

FSStat

Concurrent and Distributed Programming
a.y. 2025–2026

Buda Marco

Merighi Daniele

Turchi Jacopo

June 10, 2026

1 Asynchronous Programming based on Event Loops

This version implements `FSStatLib` on the Node.js runtime, whose concurrency model is a single-threaded *event loop* driving non-blocking I/O. The design adheres strictly to this discipline: all asynchrony is expressed through `Promises` and the `async/await` syntax, and no worker threads are introduced. The public method has the signature `getFSReport(D, MaxFS, NB)` and returns a `Promise` that resolves to the report, i.e. the total file count and the $NB + 1$ size bands.

1.1 Problem Analysis

From a concurrent standpoint, producing the report is an **I/O-bound** problem rather than a CPU-bound one. The actual computation, classifying each file size into a band, is negligible. The cost is dominated by filesystem metadata operations (`readdir` on directories, `stat` on files), each of which is a request to the operating system that completes after some latency.

This observation reframes the goal of concurrency. There is no heavy calculation to spread across cores: a purely sequential traversal would spend most of its time *idle*, blocked waiting on the disk or the OS cache. The opportunity is instead to keep many I/O operations *in flight at the same time*, so that their latencies overlap and the program progresses while individual requests are pending.

Two aspects are particularly relevant for the design:

- **Unbounded parallel task generation.** The directory tree has unknown shape and depth and is discovered incrementally. A naive “launch everything at once” strategy would try to open thousands of file descriptors simultaneously, exhausting the per-process OS limit (an `EMFILE` error). The degree of concurrency must therefore be *bounded* explicitly.
- **Aggregation of shared state.** The total counter and the histogram are updated by many logically concurrent tasks. Crucially, however, the event loop executes

JavaScript callbacks *one at a time* on a single thread: there is no true parallelism, hence no data races on this shared state. Synchronisation reduces to the cooperative interleaving of tasks at `await` points, which removes the need for the monitors and locks that characterise a thread-based solution.

1.2 Strategy and Design

Because there are no threads, the program has no *active* components in the classical sense: the single control flow is the event loop itself, which dequeues and runs ready continuations. The architecture is consequently a small set of cooperating *passive* modules, illustrated in the data flow from `getFSReport` down to the leaf `stat` calls.

Parallel execution with `Promise.all`. The traversal is a recursive `walk` function that embodies a structured fork/join pattern. For a directory it reads the entries, maps each one to a `Promise`, and awaits them collectively with `Promise.all`. Sub-directories recurse, regular files are queried via `stat` and counted, and a directory's `walk` resolves only once its whole subtree has been processed.

```
async function walk(dir, ctx) {
  const entries = await ctx.semaphore.run(
    () => readdir(dir, { withFileTypes: true }));
  // fork: one branch per entry; join: await all of them
  await Promise.all(entries.map(
    (entry) => handleEntry(dir, entry, ctx)));
}
```

This shape lets the whole tree be explored concurrently: many directories and files are visited in parallel from the point of view of the program, even though, at the machine level, only the I/O is genuinely overlapped while the JavaScript runs sequentially.

Bounding concurrency with an asynchronous semaphore. To prevent the unbounded parallel task generation from exhausting file descriptors, every operation that consumes one (`readdir` and `stat`) is wrapped by an `AsyncSemaphore`, a counting semaphore adapted to the event loop. `acquire()` returns an already-resolved `Promise` when a permit is available, otherwise it returns a pending `Promise` whose `resolve` function is queued. The `release()` method then hands a permit to the next waiter, if any. The `run(task)` helper acquires, awaits the task, and releases in a `finally` block.

With a default of 64 permits, at most 64 I/O operations are ever in flight, regardless of how large the tree is. The semaphore thus acts as a flow-control mechanism: excess tasks simply queue up as pending promises and are admitted as permits are freed. This is the single most important concurrency decision of the design, as it decouples the (potentially huge) logical concurrency from the (bounded) physical concurrency that the OS can sustain.

Lock-free aggregation. The histogram and the file counter are plain passive objects mutated by the `onFile` callback. No locking is needed: thanks to the run-to-completion

semantics of the event loop, each callback body runs to the end without being interrupted, and the only suspension points are the explicit `await`s. The mutations therefore execute atomically with respect to one another, even though dozens of `handleEntry` promises are outstanding. This is a direct benefit of the cooperative, single-threaded model and stands in sharp contrast to a thread-based version, where the same aggregation would require a synchronised monitor.

Robustness. A few cases are handled to keep the scan well-behaved. Symbolic links are skipped, which avoids both traversal cycles and accidentally escaping the target subtree. The root is validated with `lstat` up front, so a non-directory argument fails fast with a clear error instead of silently yielding an empty report. Finally, errors on individual entries (e.g. permission denied) are captured in a `skippedEntries` list rather than aborting the whole traversal, so a single unreadable directory does not invalidate the report.

2 Reactive programming using ReactiveX

This version implements `FSStatLib` by conceptualizing the entire problem in terms of streams of data and events, then by programming them in Scala using the RxJava library and its standard operators. The nature of this paradigm also made it sufficiently straightforward to extend the solution to include the proposed *interactive* functionalities, as described in the assignment's **Optional point**.

The resulting public method has the signature `getFSReportInteractive(D, MaxFS, NB, stopper)`, where the returned value is a `ConnectableObservable` of reports, i.e. a *hot* stream, and `stopper` is a `CompletableSubject` providing the ability to terminate the dynamic stream of updates prematurely.

A simple Graphical User Interface was also added, so as to provide an effective way to showcase these additional functionalities.

2.1 Domain Modeling

Due to the declarative essence of reactive programming, finding a way to model the problem is enough to also get a working strategy, along with a nearly one-to-one mapping to the final implementation code. For this analysis, we adopt a series of marble diagrams as a concise way to show the overall behavior.

The problem can be broken down into two main challenges: generating a stream of files from the recursive traversal of the starting directory and processing it to obtain a stream of reports.

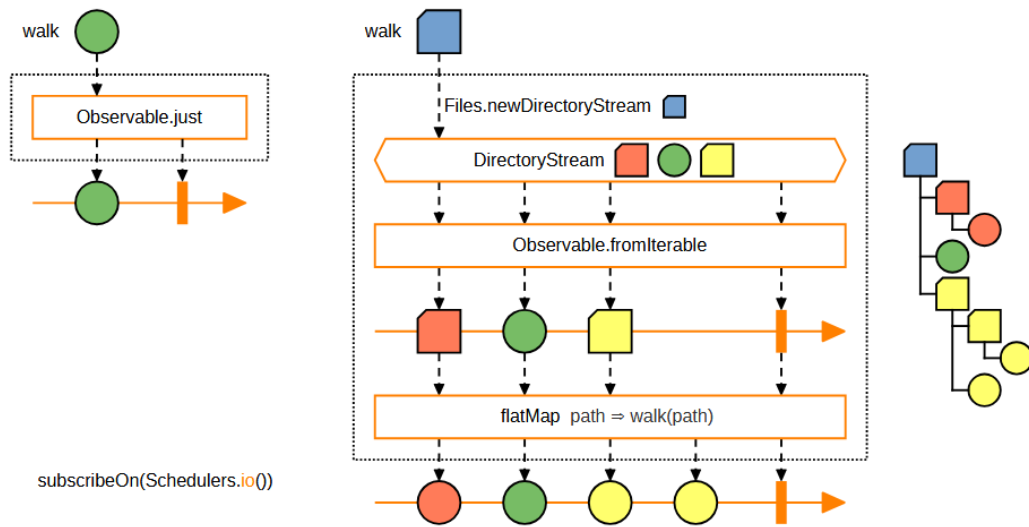


Figure 1: Generation of a stream of files from the recursive traversal of a file tree.

Figure 1 shows an example of the `walk(path)` function in action, which solves the first subproblem. When applied to a regular file, it simply returns a stream with that file as its only value. When applied to a directory, it instead calls itself recursively on each of the contained files and flattens out the results by leveraging the `flatMap` operator.

The entire process is guaranteed to be performed on designated I/O threads thanks to the call to `subscribeOn(Schedulers.io())`. This solution can also be easily made parallel by using the operator `flatMap(mapper, maxConcurrency)` provided by RxJava out of the box.

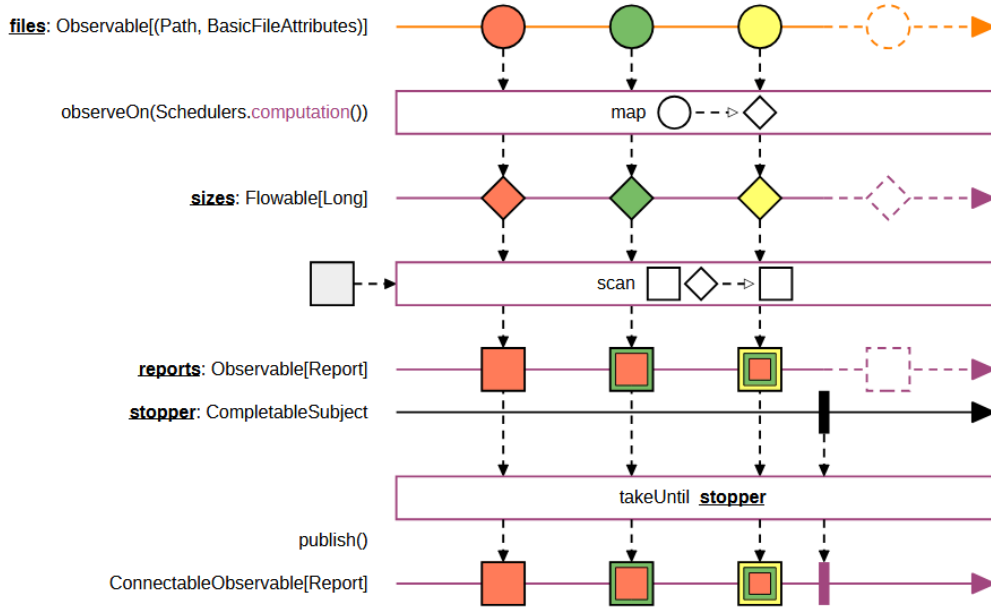


Figure 2: Processing of a stream of files into a stream of reports.

Figure 2 shows the solution to the second subproblem, which starts by extracting the file sizes using the `map` operator, then combines them into reports. Again, each report keeps track of the total number of files and the distribution between the different size bands at a given time.

The transformation from sizes to reports is done by first defining a function to update a report based on a new given file size, then by passing it to the `scan` operator, which takes care of emitting all intermediate results. All of these transformations are scheduled as computation thanks to call to `observeOn(Schedulers.computation())`.

The ability to interrupt the stream prematurely is implemented by applying the `takeUntil` operator, which disposes of the entire upstream whenever the `stopper` subject completes.

In the end, the stream of reports is published as a *hot* stream. Section 2.3 describes the full reasoning behind the choice of *cold/hot* streams at the different stages.

2.2 Graphical Interface

The included showcase program was developed using Java Swing components and maintaining the reactive programming approach. All buttons and other UI elements are configured to emit on specialized `PublishSubjects` and all visual updates are set to take place through the subscription to data or event streams, while using a custom scheduler to ensure that they are executed on the Event Dispatch Thread.

```
val EDT_SCHEDULER = Schedulers.from(SwingUtilities.invokeLater(_))
```

The interface includes a dialog to assist in the selection of the starting directory, buttons to start and stop the underlying computations, and a text area where the live statistics are shown. Figure 3 shows a snapshot of the program's execution.

The statistics are set to update no more than 10 times per second, so as to not overwhelm the user and to avoid a waste of cpu time. This is done by applying the `sample` operator with an interval of 100 milliseconds, while making sure that the last report is always emitted.

```
reports
  .sample(100, TimeUnit.MILLISECONDS, emitLast = true)
  .observeOn(EDT_SCHEDULER)
  .subscribe(...)
```

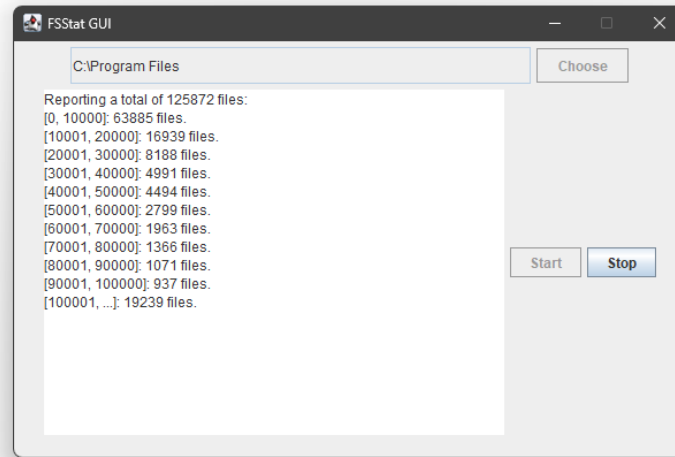


Figure 3: A capture of the program's interface.

2.3 Lazyness and Backpressure

When considering the end result, computing statistics on a directory imposes the strict requirement that no file is ever skipped (except for unavoidable I/O errors). For this reason, most of the program's computation makes use of *cold* Observables, which are lazy and operate on a pull model, thus preventing any form of backpressure.

Only at the end is the stream of reports published as a *hot* Observable. At this point, ensuring that every value gets consumed is no longer relevant. In fact, it is expected that subscribers may only want to consider updates at given time intervals, as is the case for our showcase program. In this context, the push model is clearly a better fit.

Still, no specific backpressure strategy is required, as the periodic sampling already takes care of discarding unwanted updates, motivating the usage of Observables as opposed to the more advanced Flowables.

3 Virtual Threads

This version implements `FSStatLib` using Java Virtual Threads, an implementation of lightweight java threads. The concurrency model relies on a thread-per-task paradigm, allowing high concurrency.

3.1 Problem Analysis

Traversing a file system and reading data is inherently an I/O-bound operation.

- **Blocking I/O:** Standard filesystem calls such as `Files.isDirectory` and `Files.size` block the executing thread while awaiting completion from the operating system.
- **OS Thread Limits:** In traditional Java concurrency, mapping one OS (platform) thread to each directory traversal leads rapidly to resource exhaustion due to the high memory footprint of OS thread stacks and severe context-switching overhead.
- **Virtual Thread Utility:** Virtual threads detach the thread abstraction from the underlying OS threads. Upon encountering a blocking I/O operation, the JVM unmounts the virtual thread, freeing the carrier thread to execute other virtual threads. This permits thousands of concurrent I/O operations without draining system threads.

3.2 Strategy and Design

The architecture follows a recursive fork-join pattern managed by a virtual thread executor.

Thread-per-task execution. `Executors.newVirtualThreadPerTaskExecutor()` is instantiated for the lifecycle of the task. For each subdirectory encountered during traversal, a new asynchronous task is submitted to this executor. The directory tree branches are discovered and processed concurrently.

Lock-free aggregation. Shared mutable state and explicit locking mechanisms (e.g. `synchronized` monitors) are avoided.

- Each directory processing task instantiates a local `FSReport` initialized with an empty size `distribution` array and a file counter.
- The task processes regular files sequentially within its scope, updating local counters.

- Subdirectory tasks return a `Future<FSReport>`. The parent task waits for its subtasks to complete via `task.get()` and utilizes the `FSReport.merge()` function to compute the sum of the data records.

This fork-join pattern ensures thread safety without synchronization, building the report bottom-up.

Asynchronous entry point. To prevent blocking the main thread while the executor coordinates the tree traversal, the root execution task is wrapped in a `FutureTask` and launched on a virtual thread via `Thread.ofVirtual().start(rootTask)`. The caller receives the `FutureTask` handle immediately.

```
try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {  
    return computeDirectory(Path.of(d), maxFS, nb, executor);  
} // The executor implicitly waits for all submitted tasks to complete before closing
```